Lab 7: Fuzzing for Input Validation Bugs

Student: Andey Hemanth | **SRN:** PES2UG24CS061 | **Course:** Software Engineering (UE24CS351A)

1. Objective and Tool Overview

This lab tests input validation in income_tax_parser.py using the Hypothesis fuzz testing library. Fuzzing provides pseudo-random, malformed, and boundary inputs to functions. This process exposes unhandled crashes and edge-case exceptions. We then harden all functions using defensive programming to prevent crashes.

2. Fuzzing Workflow and Terminal Screenshots

SS1: Execution of Buggy Module with Standard Inputs

The buggy module functions correctly only when inputs match exact expected formats.

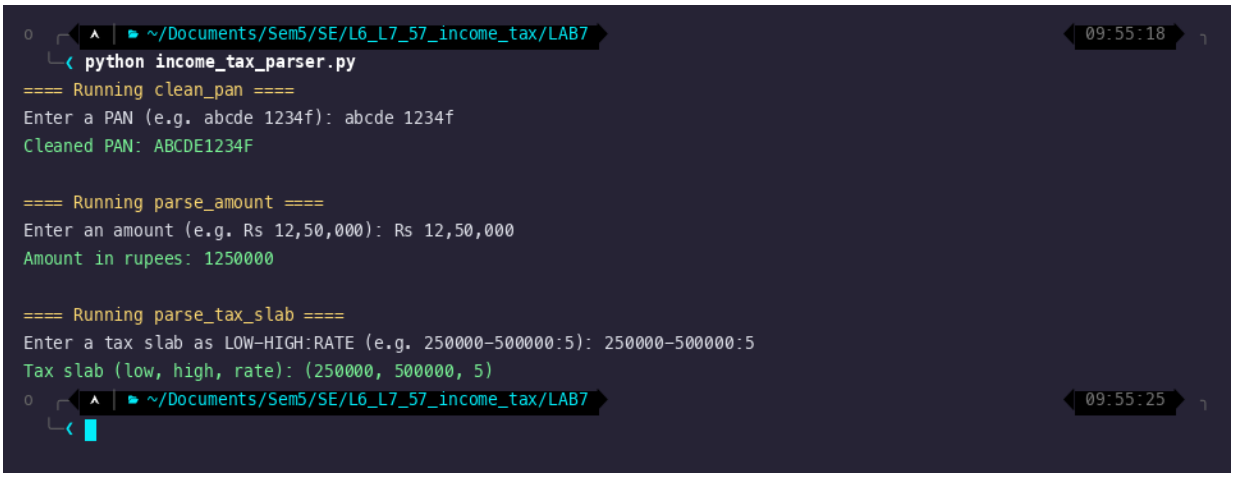


Figure 1 (SS1): python income_tax_parser.py running on the buggy module with normal inputs.

SS2: Fuzz Tests Failing on Buggy Module

Hypothesis automatically finds minimal failing inputs (e.g. None, empty string, and missing delimiters).



Figure 2 (SS2): pytest test_income_tax_parser.py failing on all 3 functions with Hypothesis counterexamples.

SS3: Execution of Fixed Module

The fixed module executes safely for valid inputs while sanitizing data properly.

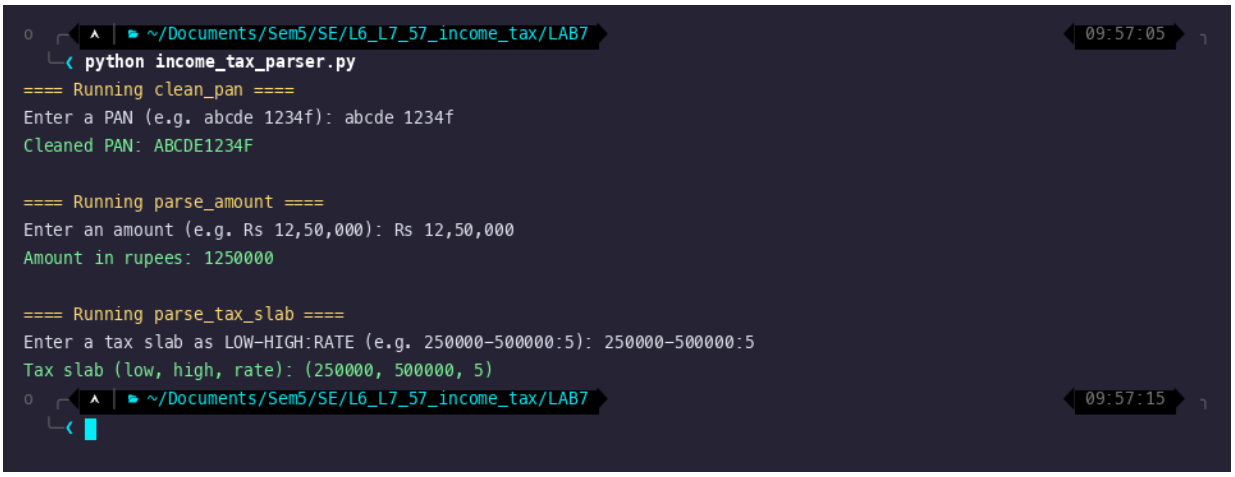


Figure 3 (SS3): python income_tax_parser.py running on the fixed module.

SS4: Fuzz Tests Passing on Fixed Module

Hypothesis executes hundreds of randomized inputs across all functions without any unhandled exception or crash.

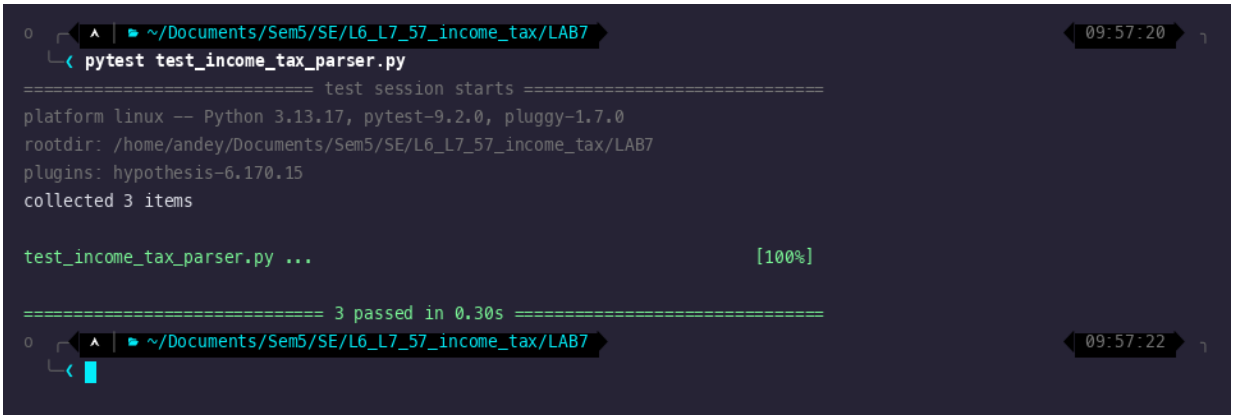


Figure 4 (SS4): pytest test_income_tax_parser.py passing 100% on the hardened module.

3. Reflection and Technical Analysis (ASD-STE100)

Question 1: What unexpected inputs caused the initial functions to crash, and what exceptions did they raise?

The initial module assumed that all inputs were valid strings in exact formats. Hypothesis detected multiple unhandled exceptions:

• **clean_pan:** Input None raised AttributeError: 'NoneType' object has no attribute 'strip'.

• **parse_amount:** Input None raised AttributeError on string replace. Input '' (empty string) raised ValueError: invalid literal for int() with base 10: ''.

• **parse_tax_slab:** Input None raised AttributeError. Inputs without delimiter ':' (e.g. '') raised ValueError: not enough values to unpack. Inputs with non-numeric values raised ValueError.

Question 2: How does property-based fuzz testing with Hypothesis compare to example-based unit testing?

Traditional unit tests check only known values chosen by the developer. Developers often miss unexpected edge cases. Hypothesis generates hundreds of random, malformed, and Unicode inputs automatically. When a test fails, Hypothesis applies input shrinking. Shrinking finds the smallest minimal input that triggers the failure (e.g. None or ''). This reduces debugging time and exposes hidden defects.

Question 3: How did defensive programming resolve these bugs, and how do they apply to production systems?

We applied three defensive techniques to fix the vulnerabilities:

1. **Type Validation:** We checked isinstance(val, str) before string operations.
2. **Structure Checks:** We verified delimiters before splitting strings.
3. **Safe Fallbacks:** We caught conversion errors and returned safe default values (e.g. '', 0, or (0, 0, 0)).

In production systems, user input is untrusted. Defensive input validation prevents unhandled crashes, service downtime, and injection attacks.